Edge Restoration after Over-smoothing

Unsharp masking (sharpening): complete explanation with examples

SIMATS Engineering | Computer Vision (ITA05) | Assessment Tool 2: Scenario-Based

Name: Keerthivasan S | **Reg No:** 192312106

1. The scenario and the questions

Scenario: After noise removal, the image looks too smooth and the edges have become blurry.

- **(a)** Why did the edges become blurry?
- **(b)** What problems come from over-smoothing?
- **(c)** Which technique restores the edges?
- **(d)** Why did you choose it?
- **(e)** Write your answer step by step.

How to use this document: Part 2 explains the basic ideas first. Parts 3 to 7 answer (a) to (e) in detail with worked numbers. Part 8 is a short exam-ready answer, and Part 9 has extra viva questions.

2. Basic ideas you must know first

2.1 What is a digital image?

A digital image is a grid of tiny squares called **pixels**. In a gray image every pixel stores one number from **0 to 255**. The number 0 means black, 255 means white, and the numbers in between are shades of gray. Dark text pixels have low numbers and bright paper pixels have high numbers.

2.2 What is an edge?

An **edge** is a place where the gray value changes suddenly from one level to another, for example from the dark ink of a letter to the bright paper. Edges give an image its shape, so we see letters, objects and borders because of edges. A sharp edge changes in one step. A blurry edge changes slowly over several pixels.

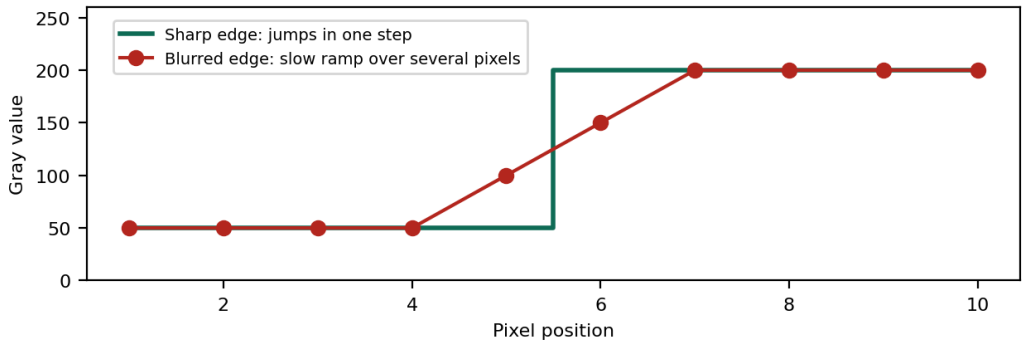


Figure 1: A sharp edge (green) jumps from 50 to 200 in one step. A blurry edge (red) climbs slowly over three or four pixels.

2.3 Frequency: edges are high frequency

Parts of an image that change **slowly** (flat paper, smooth sky) are **low frequency**. Parts that change **quickly** (edges, text, fine texture) are **high frequency**. Noise is also high frequency. A smoothing filter keeps low frequencies and weakens high frequencies, so it removes noise but also weakens edges.

Part of the image	Changes	Frequency	After smoothing
Flat paper or background	Slowly	Low	Stays the same
Edges, text, fine detail	Quickly	High	Weakened (blurry)
Noise	Randomly and quickly	High	Removed (good)

2.4 What is happening in this scenario, and what do we need?

- Noise was removed with a smoothing filter. The noise is gone, but the picture now looks soft and the edges are weak.
- We need to bring the edges back **without bringing the noise back**.
- So we need a method that finds the lost high-frequency detail and adds it back in a controlled amount. That method is **unsharp masking**.

3. Answer to (a): Why did the edges become blurry?

Answer: Smoothing filters replace each pixel with a value made from its neighbours. At an edge, the neighbours come from both the dark side and the bright side, so the edge is mixed into a slow ramp. Edges are high frequency, and smoothing removes high frequency.

Cause	What happens	Result in the image
Averaging or Gaussian filter	Each pixel becomes a weighted average of its window, so dark and bright values mix.	Edges turn into gray ramps.
Window too large (5x5, 7x7 or more)	A bigger window mixes pixels from farther away.	Stronger blur, thin lines disappear.
Filter applied many times	Each pass smooths the result of the last pass.	Blur keeps growing.
Gaussian sigma too high	The filter spreads each pixel over a wide area.	Soft, flat picture.
Median filter on thin details	Lines thinner than half the window are treated like noise and removed.	Thin strokes break or vanish.

Example at a letter edge: a 3x3 window on a letter edge has 3 dark pixels (value 10) and 6 bright pixels (value 200). The average is (3 x 10 + 6 x 200) / 9 = 1230 / 9 = **137**. This muddy gray was not in the original picture, so the edge is blurred.

4. Answer to (b): What problems come from over-smoothing?

Answer: Fine detail and edges are lost, text and objects look soft, and later steps such as edge detection, segmentation and OCR give poor results.

- **Loss of detail.** Small features, thin lines and texture become faint or disappear.
- **Text is hard to read.** Letters have soft borders and close letters merge, so an 'e' can look like an 'o'.
- **Weak edges for later steps.** Edge detectors find weaker, wider edges. Thresholding and segmentation draw object borders in the wrong place.
- **OCR and recognition errors.** Characters and features are not sharp enough to match.
- **Poor appearance.** The picture looks flat and out of focus for printing and sharing.

5. Answer to (c): Which technique restores the edges?

Answer: Use **unsharp masking** (image sharpening): subtract a blurred copy to find the edge detail, then add that detail back to the image.

5.1 Key words

- **Sharpening:** making edges steeper so that the picture looks clearer.
- **Mask:** the edge detail found by subtracting a blurred copy from the image: M = F - B.
- **Amount (k):** how strongly the mask is added back. A small k gives a gentle effect, a large k a strong effect.
- **Clipping:** keeping the final values inside 0 to 255.

5.2 Why does unsharp masking restore edges?

If we blur the image again, the blur removes the fine detail. So **image minus blurred copy** contains only the fine detail (edges). This is the mask. On the dark side of an edge the mask is negative and on the bright side it is positive. Adding k times the mask to the image makes the dark side a little darker and the bright side a little brighter. The edge becomes steeper and looks sharper. In flat areas the mask is zero, so nothing changes there.

5.3 Understanding with one edge

Figure 2 shows one edge across 8 pixels (numbers in Part 7). The gray dashed line is the ideal sharp edge, the red line is the over-smoothed edge (50, 50, 62, 100, 150, 188, 200, 200), and the green line is the result after unsharp masking. The green edge is steeper, with a small overshoot on each side.

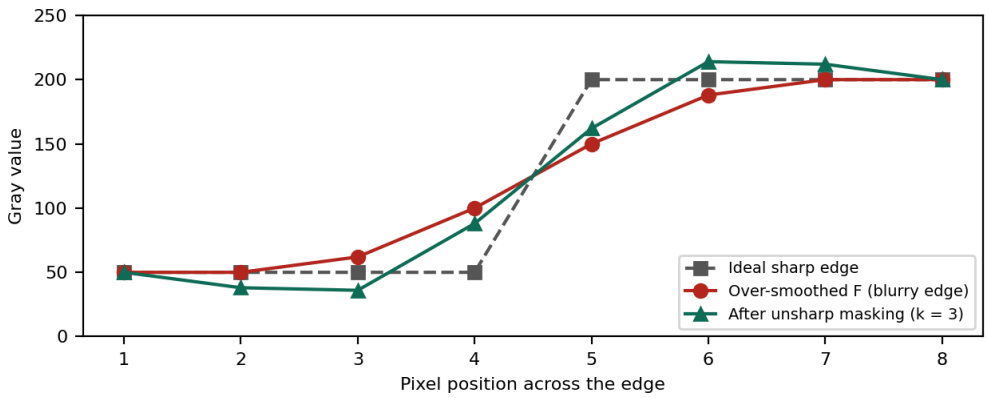


Figure 2: Red = blurry edge, green = after unsharp masking, gray dashed = ideal edge. The edge becomes steeper.

5.4 The steps of unsharp masking

Step	What to do	Why / example
1	Take the over-smoothed image F.	This is the image whose edges are weak.
2	Blur a copy of F to get B (small Gaussian or 3x3 average).	The blurred copy has even less detail than F.
3	Find the mask: M = F - B.	M holds only the edge detail. It is zero in flat areas.
4	Multiply the mask by the amount: k x M.	k controls the strength (for example k = 0.5 to 2).
5	Add it back: Output = F + k x M.	Dark side gets darker, bright side gets brighter, edge is steeper.
6	Clip the result to 0 to 255.	Keeps all values valid.

5.5 What the technique does to our image

Figure 3 shows a page of text. Panel 2 is over-smoothed, so the letters are soft. After unsharp masking (panel 3) the letters have clear borders again.

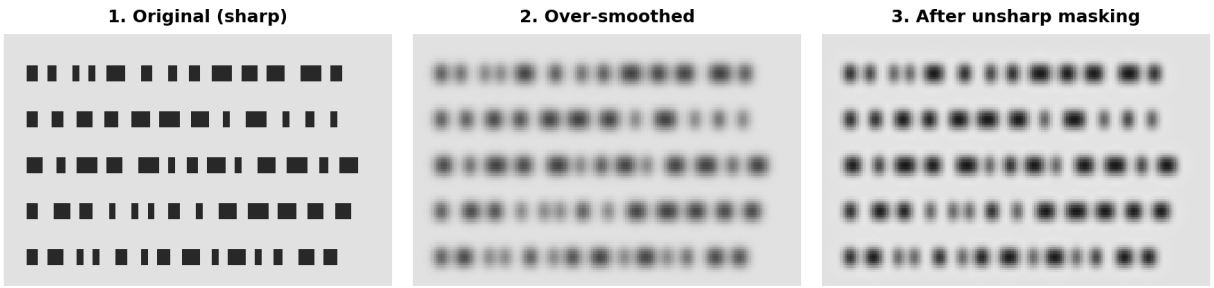


Figure 3: Original, over-smoothed, and after unsharp masking. The letters look sharp again, though not exactly as in the original.

5.6 Practical points

- **Sharpen after denoising, not before.** Sharpening also boosts noise, so remove the noise first and then sharpen gently.
- **Choose k and the blur size carefully.** Too large a k gives bright or dark halos along edges. A small blur radius (sigma about 1 to 2) sharpens fine edges, a large radius sharpens broad edges.
- **Threshold option.** Apply the mask only where |M| is above a small threshold, so flat noisy areas are not boosted.
- **Prevent the problem.** Use a small window, a median filter for impulse noise, or an edge-preserving filter such as the bilateral filter, so less blur is created.
- **Colour photos:** sharpen only the brightness channel so that colours do not shift.

In OpenCV (Python) unsharp masking is a few lines:

```
import cv2
img   = cv2.imread('smooth.png', 0)            # over-smoothed gray image
blur  = cv2.GaussianBlur(img, (0, 0), 2)       # step 2: blurred copy B
k     = 1.5                                    # amount
sharp = cv2.addWeighted(img, 1 + k, blur, -k, 0)  # F + k (F - B), clipped
cv2.imwrite('sharp.png', sharp)
```

6. Answer to (d): Why did you choose unsharp masking?

Answer: The edge detail was lost by smoothing, and unsharp masking adds exactly that detail back. It is simple, fast, easy to control with one amount k, and it works on the image we already have. Other sharpening methods are either more noise-sensitive or much more complex.

6.1 Comparison of the options

Property	Laplacian sharpening	Unsharp masking	Deblurring (Wiener, deconvolution)
Idea	Subtract the second derivative from the image	Add back image minus blurred copy	Reverse a known blur mathematically
Control over strength	Fixed (one mask)	Yes, amount k and blur size	Needs the blur model and noise level
Noise sensitivity	Very high	Moderate (blur in the mask reduces it)	Can be unstable
Simplicity and speed	Simple	Simple	Complex and slower
Needs to know the blur	No	No	Yes
Best used for	Quick strong sharpening of clean images	General sharpening after denoising	Known camera or motion blur

Why the others were not chosen: Laplacian sharpening reacts strongly to the small noise left after denoising and gives no way to tune the effect. Deblurring needs to know exactly how the blur was made, which we do not know here. Unsharp masking needs no such knowledge, and k lets us stop before halos or noise appear.

6.2 Important note

Sharpening cannot bring back detail that was completely erased. It makes the remaining edges steeper so that the picture **looks** sharp again. That is why it is best to avoid over-smoothing in the first place, and to use unsharp masking as the repair step.

7. Answer to (e): Step by step with a full example

Answer: Over-smoothed image, blur a copy, subtract to get the mask, multiply by k, add back, clip, sharper image.

7.1 Example input and output

We use one scan line of 8 pixels across an edge. The real edge went from dark (50) to bright (200) in one step, but smoothing turned it into a slow ramp. The blurred copy B is made with a 3-pixel average (the border pixel is repeated). We use an amount k = 3 so that the effect is easy to see (real photos often use 0.5 to 2).

Pixel	1	2	3	4	5	6	7	8
Ideal edge	50	50	50	50	200	200	200	200
Over-smoothed F	50	50	62	100	150	188	200	200

7.2 Detailed steps for one pixel (pixel 6, value 188)

Step 1: Take the over-smoothed value
F = 188 at pixel 6. Its neighbours are 150 (pixel 5) and 200 (pixel 7).

Step 2: Blur a copy
B = (150 + 188 + 200) / 3 = 538 / 3 = 179.3.

Step 3: Find the mask
M = F - B = 188 - 179.3 = 8.7. Positive, so pixel 6 is on the bright side of the edge.

Step 4: Multiply by the amount
k x M = 3 x 8.7 = 26.

Step 5: Add it back
Output = F + k x M = 188 + 26 = 214. The bright side becomes brighter.

Step 6: Clip to 0 to 255
214 is inside the range, so the output at pixel 6 is 214.

Figure 4: The six steps for pixel 6. The value 188 becomes 214.

7.3 Repeat for all pixels

Repeat Steps 1 to 6 for every pixel. For pixel 3 (value 62): B = (50 + 62 + 100) / 3 = 70.7, M = 62 - 70.7 = -8.7, k x M = -26, output = 62 - 26 = 36. This pixel is on the dark side, so it becomes darker. Figure 5 shows all 8 pixels.

Pixel	F	B (3-pixel average)	Mask M = F - B	k x M (k = 3)	Output
1	50	50.0	0	0	**50**
2	50	54.0	-4	-12	**38**
3	62	70.7	-8.7	-26	**36**
4	100	104.0	-4	-12	**88**
5	150	146.0	4	12	**162**
6	188	179.3	8.7	26	**214**
7	200	196.0	4	12	**212**
8	200	200.0	0	0	**200**

Figure 5: Calculation for all 8 pixels.

7.4 Check your result

- Edge strength is the largest jump between two neighbouring pixels. Before: 50 (from 100 to 150). After: 74 (from 88 to 162). The edge is about 48% steeper.
- The dark side went down (50 to 36) and the bright side went up (200 to 214). This small overshoot is what makes the edge look crisper.
- Flat parts (pixels 1 and 8) did not change, because the mask there is zero.
- The result is closer to the ideal edge, but not equal to it. Sharpening improves the edge; it does not restore lost information.

7.5 The complete process in one picture

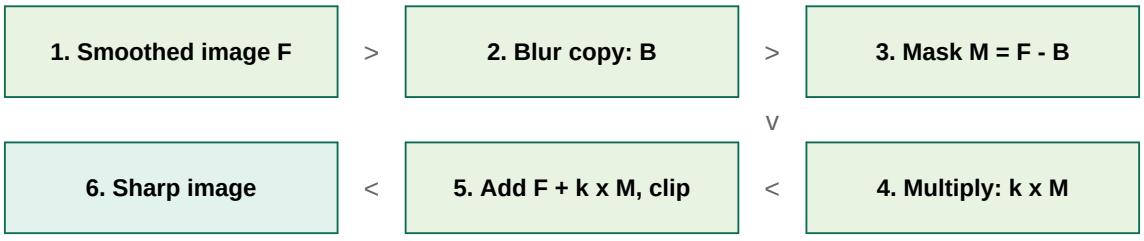


Figure 6: Flow of the unsharp masking process.

8. Short exam-ready answer

Part	Answer
(a)	Smoothing filters (averaging, Gaussian, large median) replace each pixel with a value made from its neighbours. At an edge this mixes the dark and bright sides into a slow ramp, so the edge blurs. Large windows and repeated filtering make it worse.
(b)	1. Fine detail and edges are lost. 2. Text looks soft and is hard to read. 3. Edge detection, segmentation and OCR give poor results.
(c)	Unsharp masking. Step 1: take the smoothed image F. Step 2: blur a copy to get B. Step 3: mask M = F - B. Step 4: multiply by amount k. Step 5: output = F + k x M. Step 6: clip to 0 to 255.
(d)	It adds back exactly the edge detail that smoothing removed. It is simple, fast and controlled by one amount k. Laplacian sharpening is too noise-sensitive, and deblurring needs the blur to be known.
(e)	Smoothed image, blur a copy, subtract to get mask, multiply by k, add back, clip, sharp image.
Example	Edge line F: 50, 50, 62, 100, 150, 188, 200, 200. With k = 3 the output is 50, 38, 36, 88, 162, 214, 212, 200. The steepest step grows from 50 to 74.
Conclusion	Unsharp masking is a good choice for restoring edges after noise removal because it brings back edge detail in a controlled amount.

9. Extra viva questions

Question	Short answer
Why subtract the blurred copy?	Because the blurred copy has only low frequencies. Image minus blurred copy leaves the high frequencies, which are the edges.
Can sharpening recover lost detail?	Not fully. It makes remaining edges steeper. Detail that was completely erased cannot be recovered.
What happens if k is too large?	Halos (bright and dark outlines) appear along edges and noise is boosted.
Why sharpen after denoising?	Sharpening amplifies noise, so noise should be removed first. Then sharpen gently.
What is high-boost filtering?	Unsharp masking with a large amount: Output = (A - 1) x F + (F - B), where A above 1. It keeps some of the original and boosts the detail.
Is unsharp masking a linear filter?	Yes. F + k(F - B) is a weighted sum of pixels, so it is linear. Only the final clipping is non-linear.
How does it differ from Laplacian sharpening?	Laplacian uses the second derivative. Unsharp masking uses the difference between the image and its blur, which is less noise-sensitive and has an adjustable radius.
How can over-smoothing be avoided?	Use a smaller window, a median filter for salt-and-pepper noise, or an edge-preserving filter such as the bilateral filter.
Which frequency-domain filter is similar?	A high-pass or high-frequency emphasis filter.